AI시대, 차세대 클라우드의 새로운 비전과 기술

openKcloud DevDay 2025

과제명: AI 반도체 클라우드 플랫폼 구축 및 최적화 기술 개발

Kubernetes에서 운영 비용 최적화 스케줄러 개발

한국전자기술연구원 (KETI)

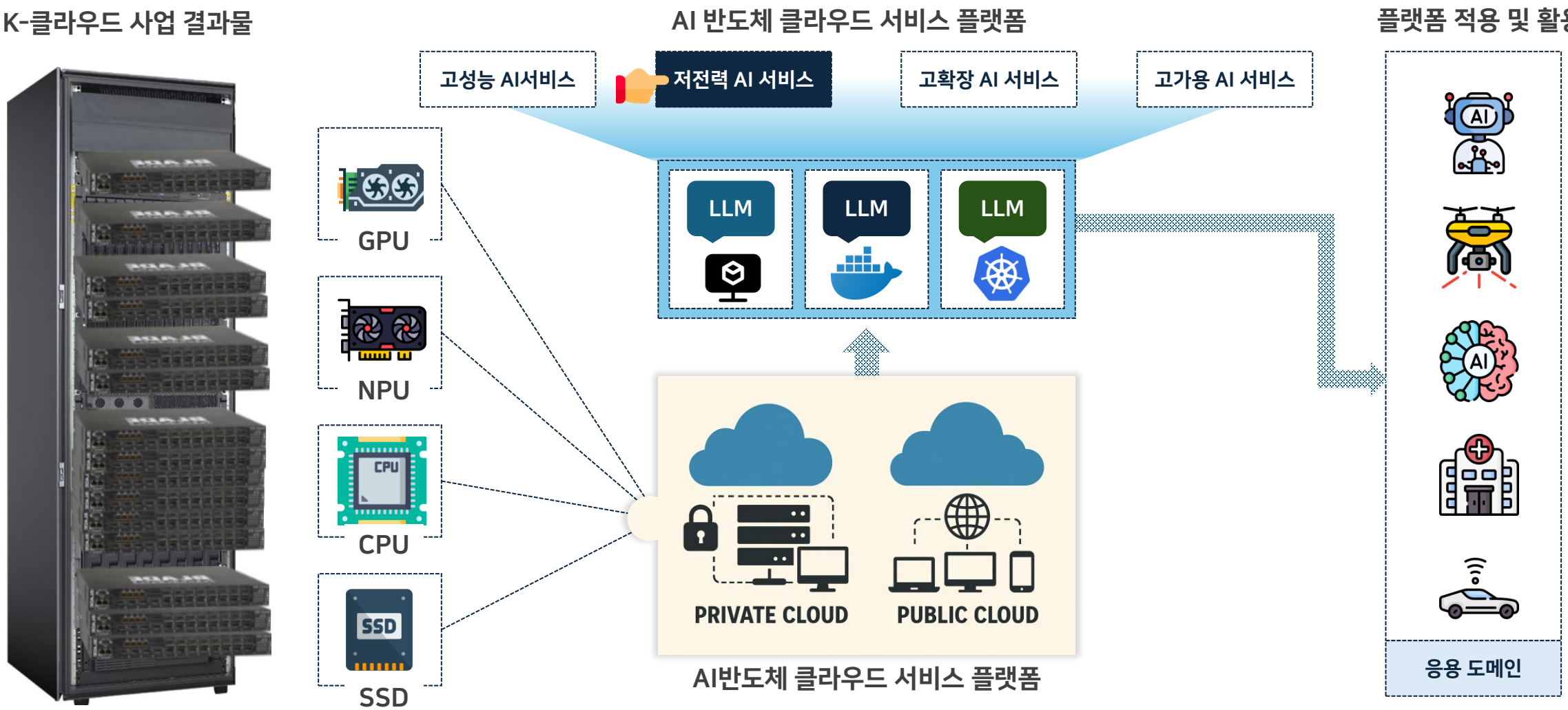
안재훈 팀장

목 차

1. 연구 개발 배경 및 개요
2. 연구 개발 환경
3. 연구 개발 내용
4. 마무리

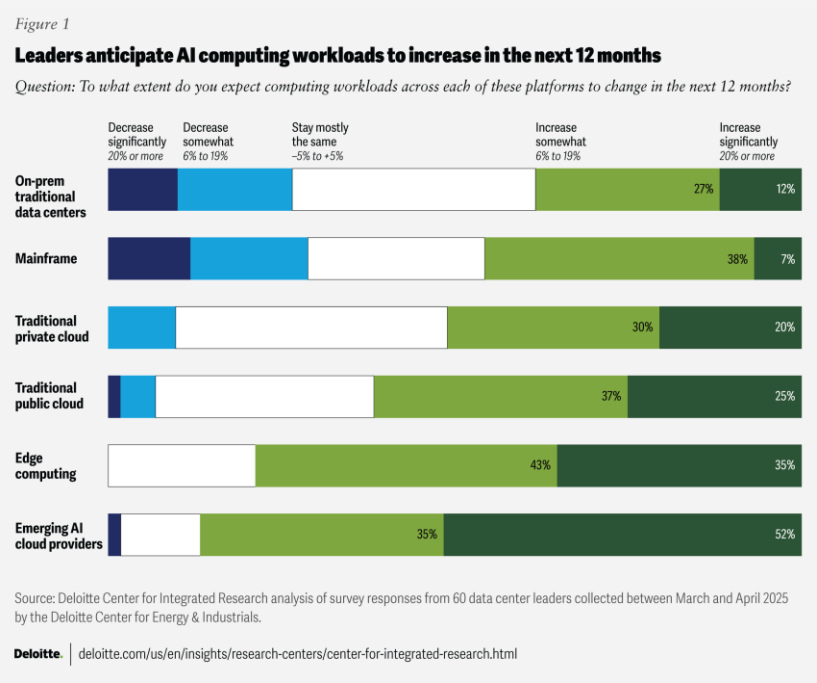
1. 연구 개발 배경 및 개요

AI반도체 클라우드 플랫폼 기술 저전력 AI 서비스 제공

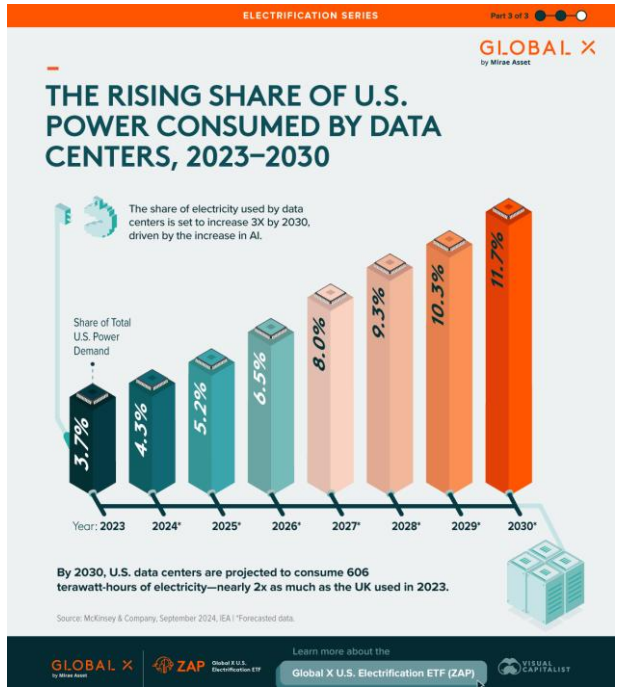


운영 비용 최적화 기술 적용의 필요성 하드웨어 인프라 확장의 한계

- 초거대 AI 모델 서비스 증가로 인한 전력·운영 비용 급증은 하드웨어 증설 + 운영 정책 기반 해결 필요
- Global CSP들은 워크로드 스케줄링 및 자동화 기반의 비용 절감 진행하였으나 "시간대별 부하분산"기술에 국한됨



❖ 거의 모든 인프라 플랫폼에서 AI 워크로드가 크게 증가



❖ 하드웨어의 증설만으로는 늘어나는 수요
→ 2030 기준 소비전력 3x 증가
→ 그에 따른 SW 최적화 필요

CSP Company	tactics	effect
Google	지역-시간대별 탄소/비용 기반 스케줄링	탄소 및 비용 절감
Microsoft Azure	시간대별 비용/부하 기반 자동 스케줄링	운영비 절감, 자원 최적화
Amazon AWS	Spot + AI 기반 스케줄링	최대 90% 비용 절감

Table. 워크로드 스케줄링 기반 전력 및 비용 절감 사례

❖ 글로벌 CSP 기업들의 흐름이 하드웨어 증설 뿐만 아니라 운영 최적화의 흐름이 추가됨

❖ 스케줄링·자동화·탄소 인지형 배치 등 "운영 기술 " 로 비용과 전력을 절감하는 사례 확인

Kubernetes 스케줄러의 한계 비용 및 절감 알고리즘 부재

Kubernetes 스케줄링 알고리즘(21종)

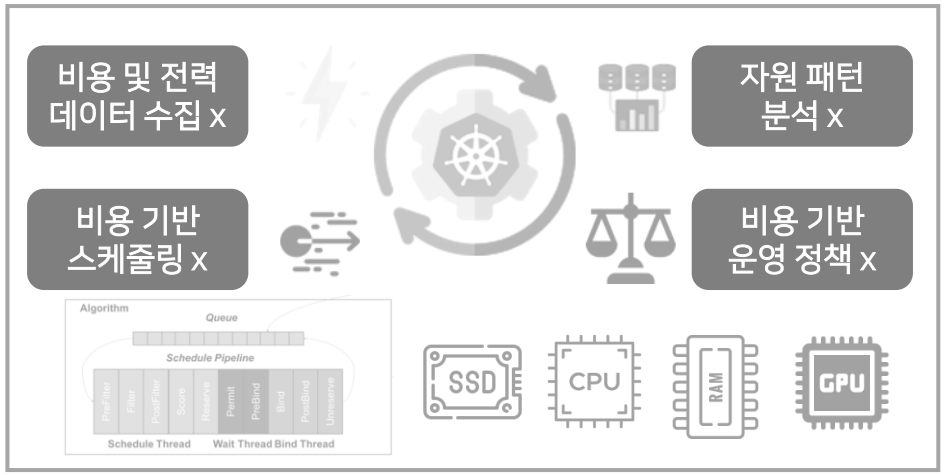
ImageLocality	VolumeRestrictions
TaintToleration	VolumeZone
NodeName	NodeVolumeLimits
NodePorts	EBSLimits
NodeAffinity	GCEPDLimits
PodTopologySpread	AzureDiskLimits
NodeUnschedulable	InterPodAffinity
NodeResourcesFit	PrioritySort
VolumeBinding	DefaultBinder
NodeResources BalancedAllocation	DefaultPreemption
	CinderLimits

→ 비용 및 전력 절감을 위한 알고리즘 부재

- 기존 Kubernetes 스케줄링은 모델 특성과 가속기 특성을 고려하지 않음
- 비용 관점 최적화 기반 클라우드 플랫폼 정책 운영 기술의 부재

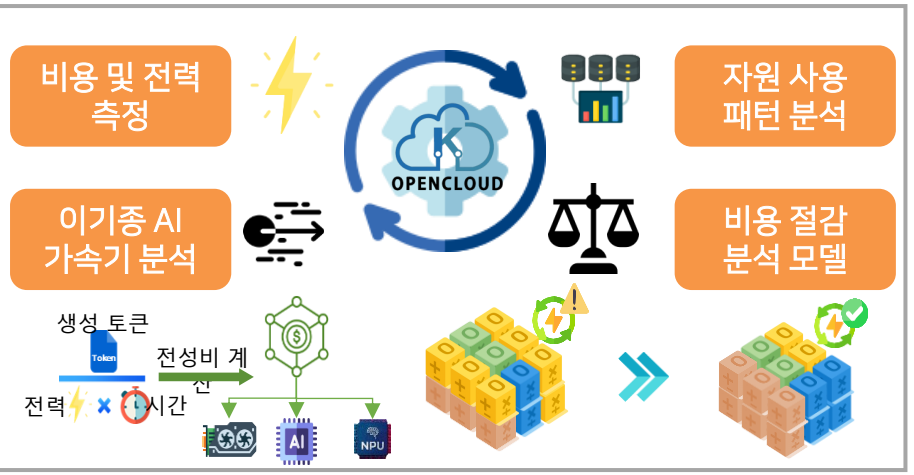
기존 기술 한계

- 운영 비용 최적화 기반 플랫폼 운영 정책 미비
- 전력 최적화 기반 스케줄링 알고리즘 부재

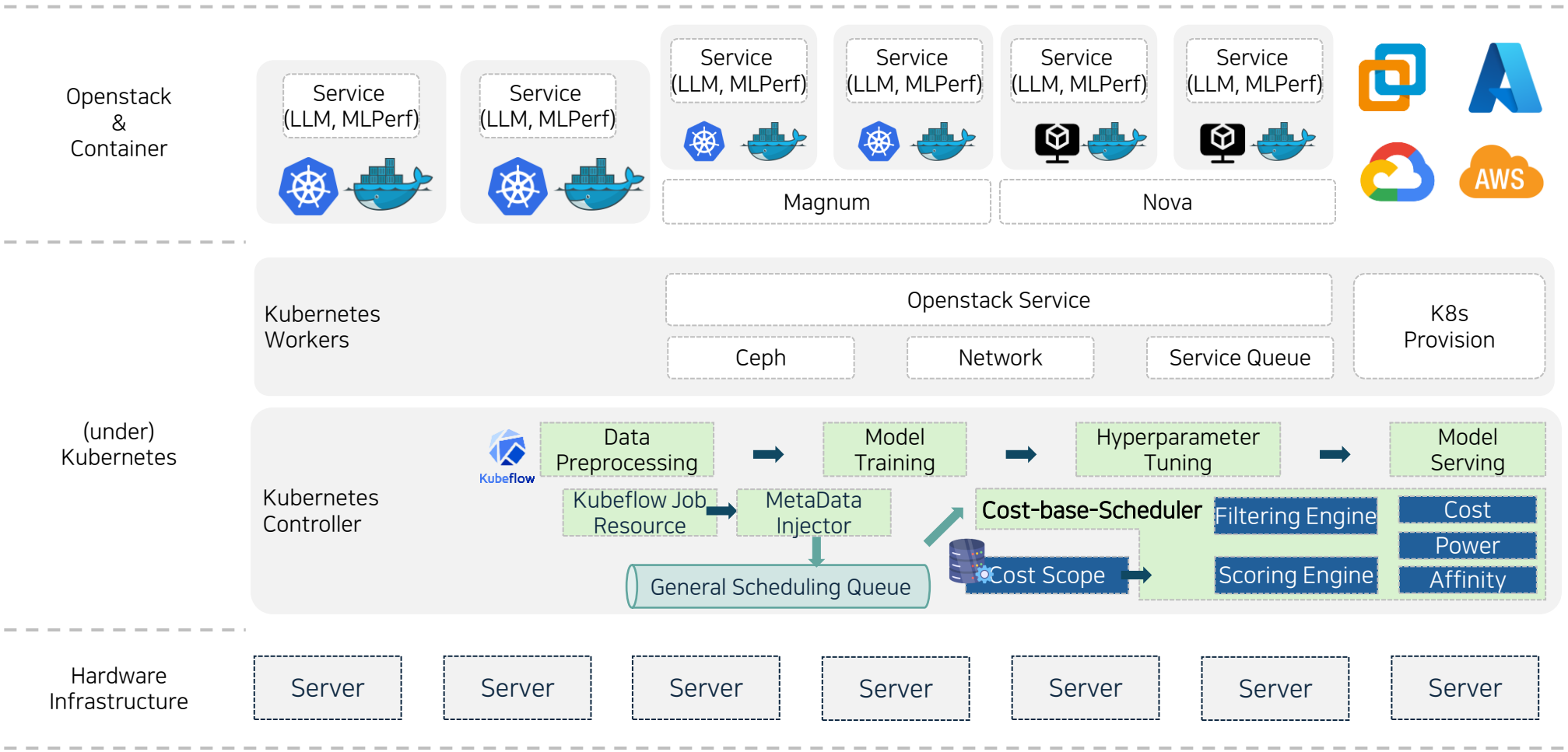


운영 비용 최적화 스케줄링

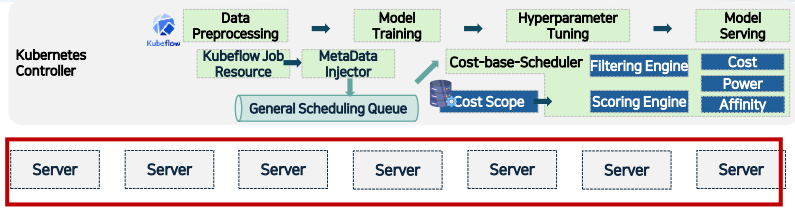
- AI 워크로드 기반 비용 최적 운영 정책 제공
- 비용 예측 및 전력 최적화 파이프라인 설계



운영 비용 최적 스케줄러 구조 AI반도체 클라우드 플랫폼 환경



2. 연구 개발 환경



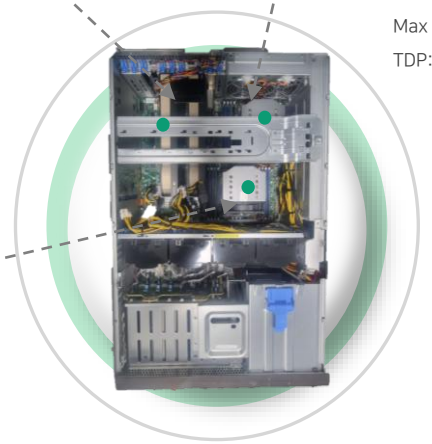
1차년도 연구 개발 환경 KETI 테스트 베드

Master/ Worker Node #1 with GPU

- Kubernetes: v1.34.1
- CNI Plugin: calico v3.27.3
- Ubuntu 24.04.2 LTS
- Container Runtime: containerd v1.7.8

GPU : NVIDIA RTX A 6000 × 2
CUDA Core : 10,752
Memory : 48 GB GDDR6
FP16 : 77.4 TFLOPS
TFLOPS : 309 TFLOPS
TDP : 300W

CPU: Intel(R) Xeon(R) Silver 4216
Core : 16
Base Clock : 2.10 GHz
Max Clock: 3.20 GHz
TDP: 100W



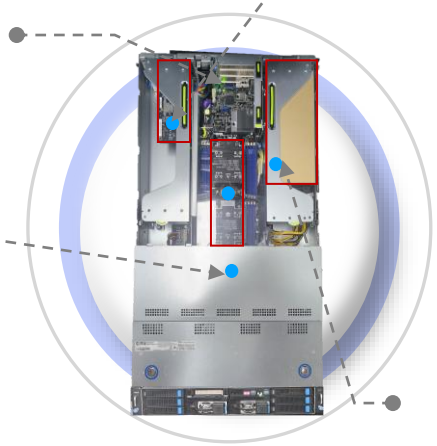
Other
Memory : 128GB DDR4
Storage : SSD 1TB, HDD 4TB
Mainboard: SupermicroX11DPG-QT
BMC : Aspeed AST2500

Worker Node #2 with GPU · NPU

- Kubernetes: v1.34.1
- CNI Plugin: calico v3.27.3
- Ubuntu 22.04.3 LTS
- Container Runtime: containerd v1.6.21

CPU: INTEL XEON SILVER 4510× 1
Core : 12
Base Clock : 2.40 GHz
Max Clock: 4.10 GHz
TDP: 150W

NPU: FuriosaAI Warboy × 1
CUDA Core : 24,064
Memory : 32 GB GDDR7
INT8 : 64 AI TOPS
TDP : 45W

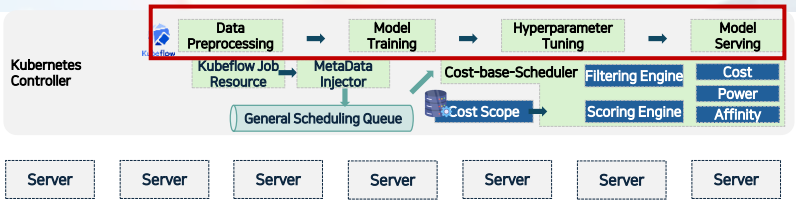


Other
Memory : 128GB DDR5
Storage : SSD 4TB
Mainboard: ASUSTek Z13PG-D16 V2
BMC : Aspeed AST2600

GPU : NVIDIA RTX PRO 6000 × 2
CUDA Core : 24,064
Memory : 96 GB GDDR7
FP16 : 4,000 AI TOPS
TFLOPS : 380 TFLOPS
TDP : 600W

kubectl get nodes –o wide

```
root@gpu-server-01:/root/kcloud_split_workload/deepspeed-tp-docker# kubectl get nodes -o wide
NAME                STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION       CONTAINER-RUNTIME
gpu-npu-server-02   Ready    worker          45h   v1.34.1   10.0.4.232    <none>        Ubuntu 22.04.3 LTS   5.15.0-160-generic   containerd://1.6.21
gpu-server-01       Ready    control-plane   46h   v1.34.1   10.0.4.230    <none>        Ubuntu 24.04.2 LTS   6.14.0-33-generic    containerd://1.7.8
```



OPENCLOUD

Kubeflow 적용 AI 다중 워크로드 검증용 운영 환경 구성

- Kubeflow Notebooks
- Kubeflow Pipelines
- Central Dashboard
- Model Registry

Kubeflow

- Kubeflow Trainer
- Cache-server
- Katib
- KServe

Central dashboard

- Kubeflow 전체 기능에 접근하는 웹 기반 중앙 포털
- Notebooks, Pipelines, Katib, KServe등 접근

Kubeflow Pipeline

- ML 파이프라인 실행·스케줄링·Run 관리 기능 제공
- Pipeline API, Scheduler, Visualization 정상 확인

Cache-server

- 파이프라인 스텝간 중복 연산 방지 및 이전 실행 결과를 재사용 하기 위한 캐시 서버 정상 동작 확인

Metadata-*

- Pipeline 실행 정보(run), artifact 메타데이터, Lineage(데이터 계보)를 저장 조회하는 백엔드 동작

kubectl get pods –n kubeflow

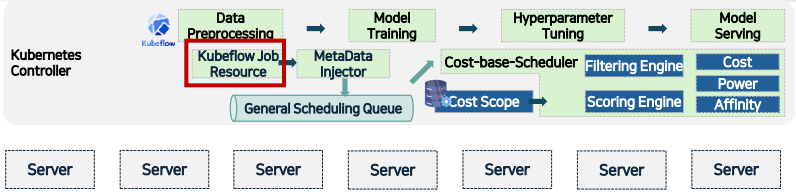
```
root@gpu-server-01:/root/kcloud_split_workload/deepspeed-tp-docker# kubectl get pods -n kubeflow
NAME                                                     READY   STATUS    RESTARTS      AGE
admission-webhook-deployment-85b5bc6d58-286pp                    Running   0             27h
cache-server-68bfdc74dd-7nghm                                    Running   0             27h
centraldashboard-646ff55c4b-9q4z7                                          0             27h
jupyter-web-app-deployment-776dcd8959-16wwb                                0             27h
katib-controller-958c7cd4d-nh2k4                         1/1     Running   0             27h
katib-db-manager-7685c45457-8wbxs                        1/1     Running   0             27h
katib-mysql-598d98b6cf-m9tg6                             1/1     Running   0             27h
katib-ui-7847548879-9dhl9                                2/2     Running   0             27h
kserve-controller-manager-7d4684cbbc-lcf6m               2/2     Running   0             27h
kserve-localmodel-controller-manager-5bcdcb7d76-wrfxp    2/2     Running   0             27h
kserve-models-web-app-fc6db6666-qrtkh                    2/2     Running   0             27h
kubeflow-pipelines-profile-controller-7d8b4c569b-sbgpf   1/1     Running   0             27h
metacontroller-0                                         1/1                               27h
metadata-envoy-deployment-6d8dc6544c-krz58               1/1                               27h
metadata-grpc-deployment-54654fc5bb-kh6v6                2/2     Running   3 (27h ago)   27h
metadata-writer-58c78fb-nlrph                            2/2     Running   0             27h
minio-556dfd4dc6-f6jbf                                   2/2     Running   0             27h
ml-pipeline-9657f9d85-qlz55                              2/2     Running   0             27h
ml-pipeline-persistenceagent-6567bc6fc4-wdgjc                    Running   0             27h
ml-pipeline-scheduledworkflow-5877957f66-tdkkf           2/2     Running   0             27h
ml-pipeline-ui-796fbd5d5d-194cb                          2/2     Running   0             27h
ml-pipeline-viewer-crd-565554fdc6-f6bbl                  2/2     Running   0             27h
ml-pipeline-visualizationserver-7c8c696bdf-j9vg7         2/2     Running   0             27h
mysql-65f957f656-65sq5                                   2/2     Running   0             27h
notebook-controller-deployment-6ddb9bcd4b-wwpmb          2/2     Running   0             27h
profiles-deployment-5b4df8484d-sjrsd                     3/3     Running   0             27h
pvcviewer-controller-manager-566c84dd58-l6qcm            3/3     Running   0             27h
tensorboard-controller-deployment-7f6d997fc9-96h67       3/3     Running   0             27h
tensorboards-web-app-deployment-bf9955d9b-qjm2w          2/2     Running   0             27h
training-operator-7958cbcdc7-b4z66                       1/1     Running   0             27h
volumes-web-app-deployment-7986764568-wvwbn              2/2     Running   0             27h
workflow-controller-58c5f6c597-rf95v                     2/2     Running   0             27h
```

cache-server

centraldashboard

metadata

ml-pipeline



Kubeflow 워크로드 타입 워크로드 특성 정의

Kubeflow 기반 선택 가능한 Custom Resource 종류

Pytorch Job

➢ 분산 학습 지원

- 동일한 성능군 Worker 배치
- 전성비 기반 리소스 선택
- Job 완료 시간 예측
- Node 간 대역폭 고려

Worker 균등 배치 + 전성비

Batch Job

➢ 단일 Pod 또는 단기성 작업

- Idle한 NPU/Edge GPU 활용
- 저비용 노드 우선 배치
- 스케일 아웃 방식으로 동적 조절

저비용 자원(NPU) 집중

Tensorflow Job

➢ Distributed training 지원

- 저비용 CPU 노드 배치
- Worker만 GPU/NPU 배치
- Worker 수 증가에 따라 HW 조합 재설정

PS(Parameter Server) 저비용, Worker는 고성능

Serving Job

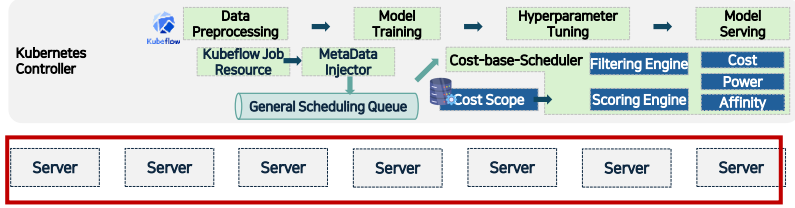
➢ 실시간 추론

- Latency 최적 디바이스 선택
- Cold start 고려한 배치
- Autoscaler 연동 실시간 조정

Latency-aware 디바이스 라우팅

Pytorch Job CRD yaml

```yaml
apiVersion: kubeflow.org/v1
kind: PyTorchJob
metadata:
  name: llm-training-pytorch-cost-01
  namespace: keti
spec:
  pytorchReplicaSpecs:
    Worker:
      restartPolicy: OnFailure
      template:
        metadata:
          labels:
            app: llm-training
            model: gpt2-small
            workload-type: training
          annotations:
            # Workload metadata for cost-base-scheduler
            workload-type: "training"
            workload.model-name: "gpt2-small"
            scheduler.affinity-mode: "preferred"
            workload.batch-size: "16"
            workload.dataset-size: "6000"
            workload.epochs: "3"
        spec:
          schedulerName: cost-base-scheduler
          containers:
          - name: pytorch
            image: nvcr.io/nvidia/pytorch:25.04-py3
            resources:
              requests:
                keti.com/gpu: 1
                memory: "4Gi"
                cpu: "2"
```



Device Plugin 세팅 이기종 가속기 k8s 리소스 등록

[GPU] nvidia-device-plugin 정보 확인

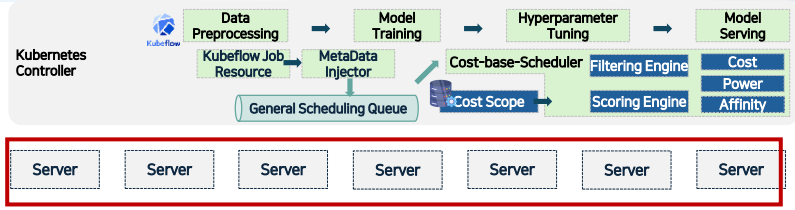
```
Name:           nvidia-device-plugin-daemonset
Namespace:      kube-system
Selector:       name=nvidia-device-plugin-ds
Node-Selector:  kubernetes.io/os=linux
Labels:         <none>
Annotations:    deprecated.daemonset.template.generation: 1
Desired Number of Nodes Scheduled: 2
Current Number of Nodes Scheduled: 2
Number of Nodes Scheduled with Up-to-date Pods: 2
Number of Nodes Scheduled with Available Pods: 2
Number of Nodes Misscheduled: 0
Pods Status:  2 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  name=nvidia-device-plugin-ds
  Containers:
   nvidia-device-plugin-ctr:
    Image:      nvcr.io/nvidia/k8s-device-plugin:v0.15.0
```

[NPU] furiosa-device-plugin 정보 확인

```
Name:           furiosa-device-plugin
Namespace:      kube-system
Selector:       app.kubernetes.io/instance=furiosa-device-plugin,app
Node-Selector:  <none>
Labels:         app.kubernetes.io/instance=furiosa-device-plugin
Desired Number of Nodes Scheduled: 1
Current Number of Nodes Scheduled: 1
Number of Nodes Scheduled with Up-to-date Pods: 1
Number of Nodes Scheduled with Available Pods: 1
Number of Nodes Misscheduled: 0
Pods Status:  1 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app.kubernetes.io/instance=furiosa-device-plugin
           app.kubernetes.io/name=furiosa-device-plugin
  Containers:
   furiosa-device-plugin:
    Image:      docker.io/furiosaai/furiosa-device-plugin:2025.3.0
```

[GPU] nvidia-device-plugin, [NPU] furiosa-device-plugin 배포 확인

```
(base) root@gpu-server-01:~/scheduler/cost_base_scheduler# kubectl get pod -n kube-system -o wide | grep plugin
furiosa-device-plugin-d9m8l            1/1     Running   2 (59m ago)   2d20h   10.244.1.111   gpu-npu-server-02
nvidia-device-plugin-daemonset-4czcp   1/1     Running   0             8m59s   10.244.1.121   gpu-npu-server-02
nvidia-device-plugin-daemonset-tw549   1/1     Running   0             11m     10.244.0.121   gpu-server-01
```



OPENCLOUD

GPU MIG 인스턴스 확인 운영 비용 기반 세부 자원 관리

NVIDIA RTX PRO6000 GPU에 MIG 인스턴스 활성화

```
(base) root@gpu-npu-server-02:~# sudo nvidia-smi -mig 1
Enabled MIG Mode for GPU 00000000:2A:00.0
Enabled MIG Mode for GPU 00000000:3E:00.0
```

MIG 인스턴스 생성

```
(base) root@gpu-npu-server-02:~# sudo nvidia-smi mig -cgi 14,14,14,14 -C -i 0
Successfully created GPU instance ID  3 on GPU  0 using profile MIG 1g.24gb (ID 14)
```

```
(base) root@gpu-npu-server-02:~# sudo nvidia-smi mig -cgi 14,14,14,14 -C -i 1
Successfully created GPU instance ID  3 on GPU  1 using profile MIG 1g.24gb (ID 14)
```

생성된 MIG 인스턴스 확인

```
(base) root@gpu-npu-server-02:~# nvidia-smi mig -lgi
+-------------------------------------------------------+
| GPU instances:                                        |
| GPU   Name             Profile  Instance   Placement  |
|                          ID       ID       Start:Size |
|=======================================================|
|   0  MIG 1g.24gb          14        3          0:3    |
|   0  MIG 1g.24gb          14        6          9:3    |
+-------------------------------------------------------+
|   1  MIG 1g.24gb          14        3          0:3    |
|   1  MIG 1g.24gb          14        6          9:3    |
+-------------------------------------------------------+
```

K8s내 MIG 인스턴스 노출을 위한 nvidia-device-plugin yaml 설정

```
      containers:
      - image: nvcr.io/nvidia/k8s-device-plugin:v0.15.0
        name: nvidia-device-plugin-ctr
        env:
        - name: MIG_STRATEGY
          value: "mixed"
        - name: FAIL_ON_INIT_ERROR
          value: "false"
        - name: NVIDIA_VISIBLE_DEVICES
```

MIG_STRATEGY 환경변수 설정

K8s에 노출된 GPU MIG 인스턴스 확인

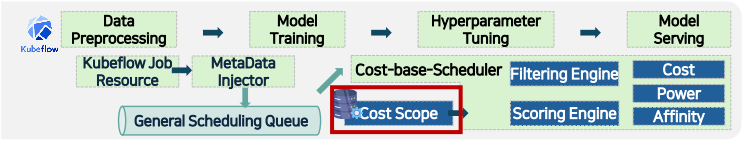
kubectl describe node gpu-npu-server-02

```
Addresses:
  InternalIP:  10.[redacted]
  Hostname:    gpu-npu-server-02
Allocatable:
  cpu:                    48
  ephemeral-storage:      1770063008091
  furiosa.ai/warboy:      1
  hugepages-1Gi:          0
  hugepages-2Mi:          0
  memory:                 65523816Ki
  nvidia.com/gpu:         0
  nvidia.com/mig-1g.24gb: 8
  pods:                   110
```

GPU 전체 요청 불가

MIG 인스턴스 8개 사용 가능 (GPU2개 * 파티션 4개)

3. 연구 개발 내용



OPENCLOUD

워크로드 사전 명세 수집 with spec metadata

사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

- **AI 워크로드의 운영 비용 최적 스케줄링을 위한 Spec 정보 수집**

GPU를 요청하는 Pod의 Spec Yaml

```yaml
apiVersion: kubeflow.org/v1
kind: PyTorchJob
metadata:
  name: llm-training-pytorch-cost-01
  namespace: keti
spec:
  pytorchReplicaSpecs:
    Worker:
      restartPolicy: OnFailure
      template:
        metadata:
          labels:
            app: llm-training
            model: gpt2-small
            workload-type: training
          annotations:
            # Workload metadata for cost-base-scheduler
            workload-type: "training"
            workload.model-name: "gpt2-small"
            scheduler.affinity-mode: "preferred"
            workload.batch-size: "16"
            workload.dataset-size: "6000"
            workload.epochs: "3"
        spec:
          schedulerName: cost-base-scheduler
          containers:
          - name: pytorch
            image: nvcr.io/nvidia/pytorch:25.04-py3
            resources:
              requests:
                keti.com/gpu: 1
                memory: "4Gi"
                cpu: "2"
```

PytorchJob

GPU 요청

Yaml 기반 사전 명세 정보

① 라벨 & 네임스페이스

- 워크로드 식별자, 네임스페이스 기반 우선순위 판단 가능

② 재시작 정책

- 다중 워크로드 배치 이슈를 고려한 정책 설정(실패 시 재시작 등)

③ 런타임 힌트 - 워크로드 특성 기반 적합 GPU 매칭

- 워크로드 특성 구분 및 가속기 요구 사항 추정
- 모델 하이퍼파라미터 기반 작업 규모 파악

④ 컨테이너 실행 환경 (+ 스케줄링 경로)

- 커스텀 스케줄러 지정 및 컨테이너 이미지 런타임 요구

⑤ 디바이스 요구 - Request / Limits 기반 최적 운용 자원 산정

- 하드웨어 요구사항, requests/limits 리소스 요구 확인

NPU를 요청하는 Pod의 Spec Yaml

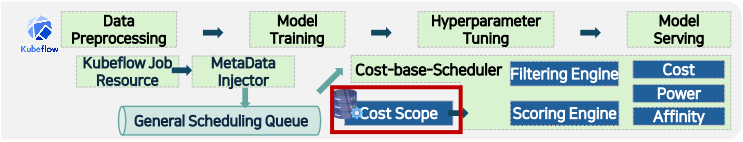
```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: warboy-vision-npu-cost
  namespace: keti
spec:
  backoffLimit: 2
  template:
    metadata:
      labels:
        app: warboy-vision
        model: yolov8n
        workload-type: inference
      annotations:
        workload.priority-class: "medium"
        workload.model-name: "yolov8n"
        scheduler.affinity-mode: "preferred"
    spec:
      restartPolicy: Never
      schedulerName: cost-base-scheduler
      containers:
      - name: warboy-vision
        image: docker.io/library/warboy-vision:latest
        imagePullPolicy: Never
        env:
        - name: FURIOSA_LOG_LEVEL
          value: "INFO"
        resources:
          requests:
            furiosa.ai/warboy: "1"
            memory: "8Gi"
            cpu: "4"
```

기본 Job

NPU 요청

OPENCLOUD

워크로드-가속기 성능 정보 워크로드 운용 효율 분석



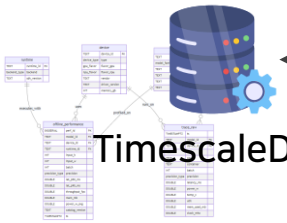
사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

- 워크로드 ↔ 가속기 특성 분석을 위한 모델 별 성능 및 전력 소비량 데이터 활용

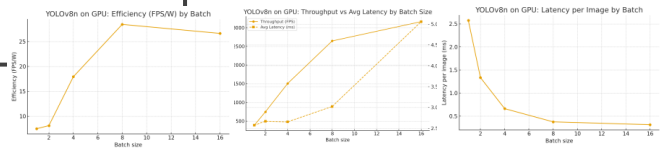
모델 수행결과 기반 카탈로그 구성

```
==============================================================================
Profile Catalog
==============================================================================

├─ yolov5l
│  ├─ RTX_6000_Pro_Blackwell
│  │  ├─ fp32
│  │  │  ├─ batch_ 1  → {throughput:  212.07 FPS, latency:   4.72ms, power:  53.47W, memory:  952.14MB}
│  │  │  ├─ batch_ 2  → {throughput:  321.12 FPS, latency:   6.23ms, power: 104.48W, memory: 1070.84MB}
│  │  │  ├─ batch_ 4  → {throughput:  417.52 FPS, latency:   9.58ms, power: 135.04W, memory: 1159.87MB}
│  │  │  ├─ batch_ 8  → {throughput:  435.64 FPS, latency:  18.36ms, power: 169.00W, memory: 1527.74MB}
│  │  │  └─ batch_16  → {throughput:  475.97 FPS, latency:  33.62ms, power: 227.19W, memory: 2213.25MB}
│  │  │
├─ yolov5l6
│  ├─ RTX_6000_Pro_Blackwell
│  │  ├─ fp32
│  │  │  ├─ batch_ 1  → {throughput:  107.42 FPS, latency:   9.31ms, power:  77.14W, memory: 1129.97MB}
│  │  │  ├─ batch_ 2  → {throughput:  283.33 FPS, latency:   7.06ms, power: 104.93W, memory: 1127.42MB}
│  │  │  ├─ batch_ 4  → {throughput:  395.74 FPS, latency:  10.11ms, power: 120.27W, memory: 1144.27MB}
│  │  │  ├─ batch_ 8  → {throughput:  412.90 FPS, latency:  19.38ms, power: 161.00W, memory: 1539.37MB}
│  │  │  └─ batch_16  → {throughput:  443.55 FPS, latency:  36.07ms, power: 211.08W, memory: 2305.35MB}
```

카탈로그 테이블 저장



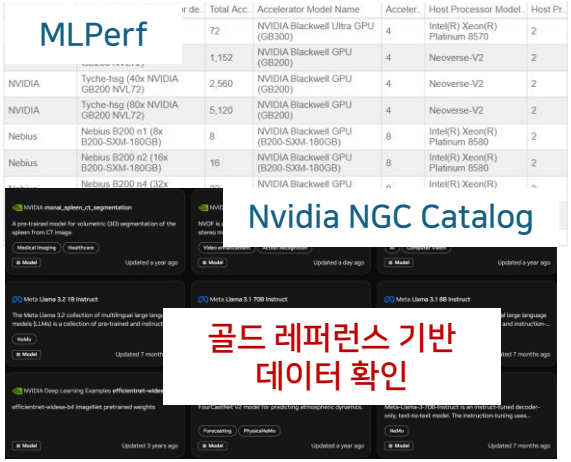
TimescaleDB



- ✓ Device Type
- ✓ Model Type
- ✓ Input Size
- ✓ Precision
- ✓ Batch Size

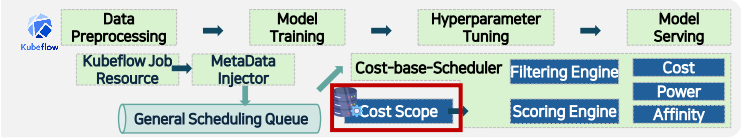
Device Type

- GPU
  - A6000
  - Pro 6000
  - H100
  - …
- NPU
  - Warboy
  - RNGD
  - …



Model	Task	Size	Accuracy
ResNet50	Image Classification	25M	75.618% (ImageNet1K-val)
EfficientNetB0	Image Classification	6.4M	72.44% (ImageNet1K-val)
EfficientNetV2-S	Image Classification	26M	83.532% (ImageNet1K-val)
SSDMobileNet	Object Detection	7.2M	mAP 0.232 (COCO 2017-val)
SSDResNet34	Object Detection	20M	mAP 0.220 (COCO 2017-val)
YOLOv5M	Object Detection	21M	mAP 0.272 (Bdd100k-val) [1]
			AP 0.284 (Bdd100k-val) [1]
			/A

Furiosa Model Zoo



OPENCLOUD

운용 효율 분석 테이블 GPU·NPU 분석 데이터 정의

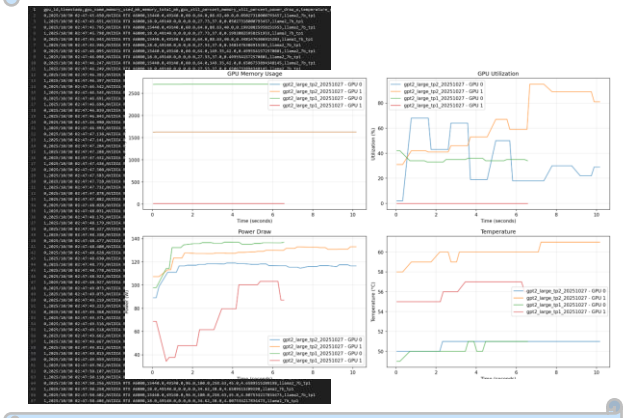
사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

비용 측정 중요 데이터
GPU 전용 데이터

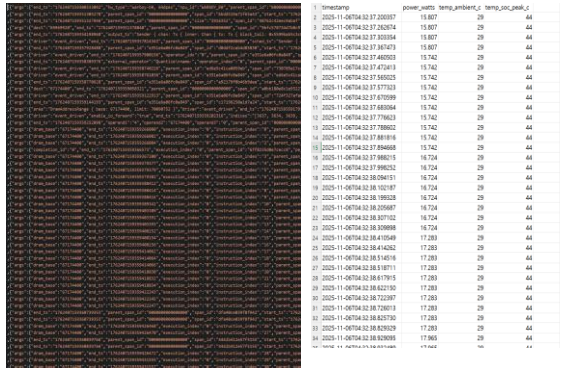
이기종 가속기 연동 기반 Trace 데이터 리스트

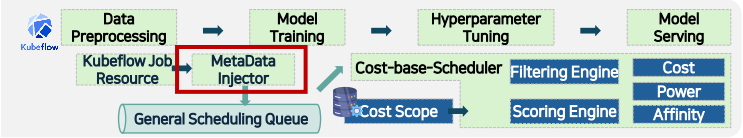
분류	대표 항목	NPU	GPU	수집 방식	비고
기본 식별 정보	device_id, uuid, chip_name, version	O	O	NPU SDK/ DCGM	장비 식별용
전력 상태	power_watts, power_limit_watts	O	O	NPU SDK/ DCGM	실시간 전력 소모
온도 센서	temp_ambient_c, temp_soc_peak_c	O	O	NPU SDK/ DCGM	GPU Core 온도
메모리 사용량	dram_usage_bytes, dram_bandwidth	O	O	NPU SDK/ DCGM	global mem, HBM
코어 사용률	npu_utilization, gpu_utilization	O	O	NPU SDK/ DCGM	SM/DPE 활용률
연산 단위당 수행 시간	task_duration, kernel_latency	O	O	NPU SDK/ DCGM	μs 단위 이벤트
온도 및 전력 트렌드	timestamp, power_watts, temp_core_c	O	O	NPU SDK/ DCGM	시계열 메트릭
오류 이벤트	thermal_throttling, memory_error, driver_fault	O	O	NPU SDK / DCGM	모니터링 및 알람 트리거
전압/클럭	voltage_v, clock_freq_mhz	-	O	- / DCGM	GPU만 측정 가능
PCIe 대역폭	pcie_tx/rx_bytes	-	O		
Tensor Core 활성화율	tensor_core_active	-	O		

GPU 기준 트레이싱 데이터 수집



NPU 기준 트레이싱 데이터 수집 현황





스케줄링 메타데이터 삽입 사전명세 분석 데이터 활용

사전명세 | 스케줄러 구조 | 스케줄링 알고리즘 | 실험 내용

기존 Pod Yaml 내 스케줄링 메타데이터 삽입

```yaml
spec:
  pytorchReplicaSpecs:
    Worker:
      restartPolicy: OnFailure
      template:
        metadata:
          labels:
            app: llm-training
            model: gpt2-small
            workload-type: training
          annotations:
            # Workload metadata for cost-base-scheduler
            workload-type: "training"
            workload.model-name: "gpt2-small"
            scheduler.affinity-mode: "preferred"
            workload.batch-size: "16"
            workload.dataset-size: "6000"
            workload.epochs: "3"
            cost.goal: "cost-optimized"
            cost.allow-spot: "true"
            cost.max-bid-rate: "medium"
            power.efficiency-preferred: "true"
            power.limit: "default"
            scheduling.priority-class: "medium"
            scheduling.deadline: "4h"
            scheduling.latency-sensitive: "false"
            scheduling.allow-shared-node: "true"
            scheduling.gpu.mig-allowed: "true"
        spec:
          schedulerName: cost-base-scheduler
          containers:
          - name: pytorch
            image: nvcr.io/nvidia/pytorch:25.04-py3
            resources:
              requests:
                keti.com/gpu: 1
                memory: "4Gi"
                cpu: "2"
```

Pod 사전명세

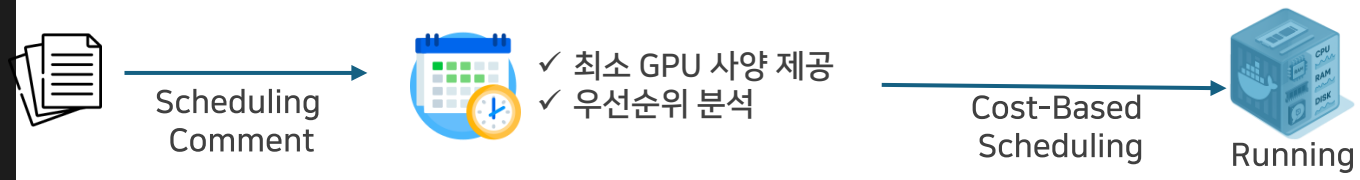
메타데이터 추가 부분

1 STEP 모델 기본 정보 확인

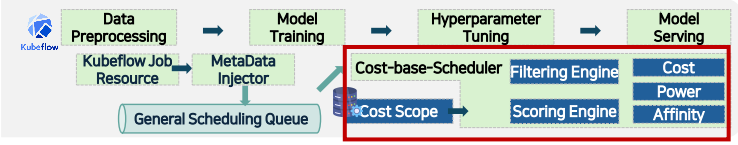
- 모델명, 작업 타입, batch size, dataset size, epochs, affinity mode 확인 가능
- 스케줄링을 위한 사전명세 카탈로그 DB의 정보를 조회하는데 사용될 수 있음

2 STEP 사전명세 기반 스케줄링 메타데이터 추가

- 비용 중심 실행(goal, allow spot, bid-rate) 기반의 spot 활용 및 비용 절감 정책 적용
- 전력 효율 기반 배치를 위한 가속기/노드 전력 정책 반영
- 우선순위 및 마감 시간을 고려한 배치 데이터 제공



Cost-base-scheduler 배포 Custom Scheduler



사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

- 기본 스케줄러(kube-scheduler)와 독립적으로 동작하는 사용자 정의 스케줄러(Custom Scheduler)등록

custom scheduler 배포 과정 (cost-base-scheduler)

```
(base) root@gpu-server-01:~# ./deploy.sh create
Applying ServiceAccount...
serviceaccount/cost-base-scheduler created
Applying ClusterRole...
clusterrole.rbac.authorization.k8s.io/cost-base-scheduler created
Applying ClusterRoleBinding...
clusterrolebinding.rbac.authorization.k8s.io/cost-base-scheduler created
Applying Deployment...
deployment.apps/cost-base-scheduler created
```

기본 kube-scheduler와 함께 구동중인 cost-base-scheduler

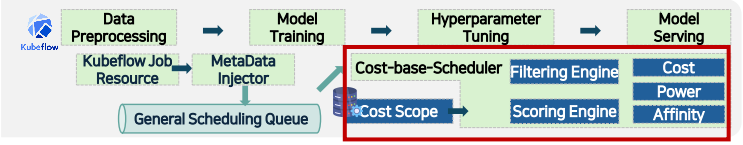
```
(base) root@gpu-server-01:~# kubectl get pod -A | grep scheduler
keti          cost-base-scheduler-79d47c46c4-fkjv8   1/1   Running
kube-system   kube-scheduler-gpu-server-01           1/1   Running
```

❖ kube-scheduler는 Control Plane에 속하는 컴포넌트로,
schedulerName을 지정하지 않은 오브젝트는 해당 스케줄러를 통해 배치됨

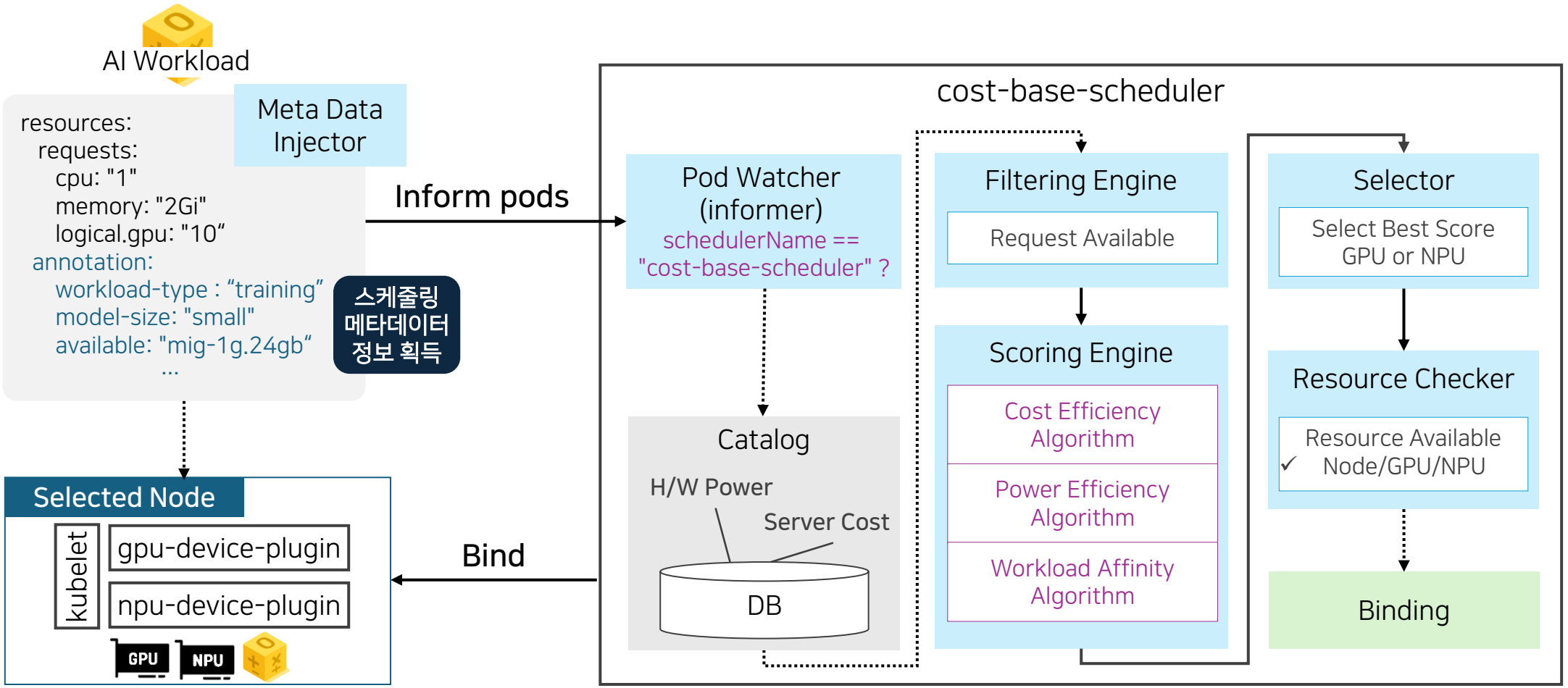
스케줄러로 cost-base-scheduler를 지정하는 pod yaml

```yaml
apiVersion: kubeflow.org/v1
kind: PyTorchJob
metadata:
  name: llm-training-pytorch
  namespace: keti
spec:
  pytorchReplicaSpecs:
    Worker:
      template:
        metadata:
          labels:
            app: pytorch-worker-gpu
        spec:
          schedulerName: cost-base-scheduler
          containers:
```

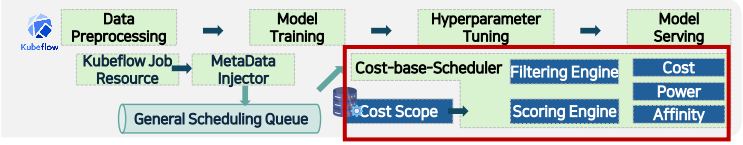
운영 비용 최적 스케줄러 cost-base-scheduler 구조



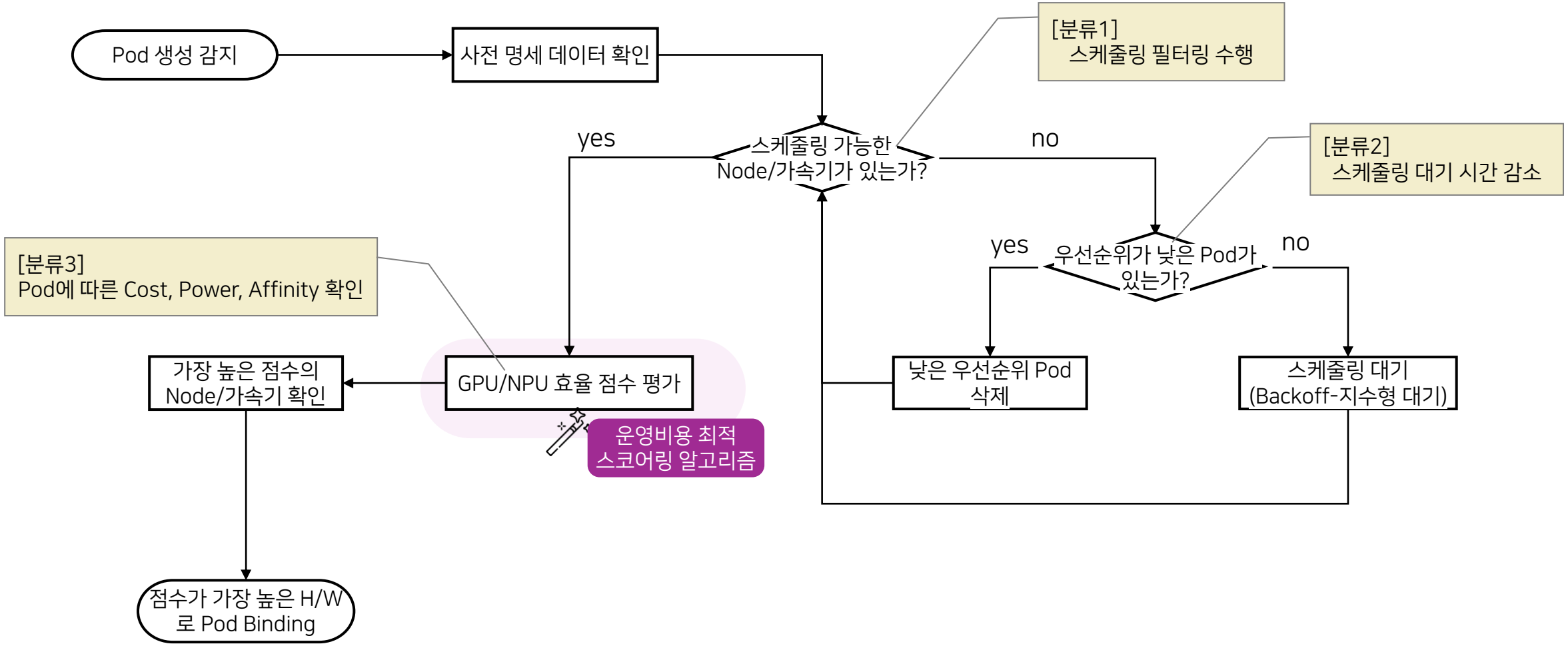
사전명세 | 스케줄러 구조 | 스케줄링 알고리즘 | 실험 내용



Cost-base-scheduler FlowChart

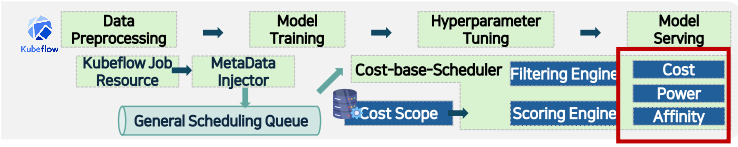


사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용



OPENCLOUD

Cost-base-scheduler 알고리즘 3종



사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

① Cost Efficiency

"운용 비용이 얼마나 드는가?"

➢ 동일한 작업을 처리할 때 가장 저렴한 가속기 선택

● Pseudocode

```
FUNCTION CalculateCostEfficiency(Node node):
  # Get Hardware Spec
  HW_Price = node.HardwareHourlyPrice
  Watts    = node.CurrentPowerWatts

  # Check MIG Instance
  IF node.IsMIG_Enabled == True:
     # Calculate MIG Instance Cost
     HW_Price = HW_Price / node.PartitionCount
     Watts    = Watts / node.PartitionCount

  Elec_Cost = (Watts / 1000.0) * ELECTRICITY_RATE
  # Calculate Total Cost Of Ownership
  Total_TCO = HW_Price + Elec_Cost
  # Cost Score
  Score = MAX_SCORE - (Total_TCO * PenaltyFactor)

  RETURN Score
```

Cost, Watt 정보 획득

MIG 가격 반영

시간당 총 소유비용 계산 및 점수 산출

② Power Efficiency

"같은 성능을 내는데 전력을 얼마나 쓰는가?"

➢ 동일한 성능을 낼 때 전력 소비가 적은 가속기를 선택

● Pseudocode

```
FUNCTION CalculatePowerEfficiency(Node node):
  # Get Watts
  DeviceWatts = node.DevicePowerWatts

  IF node.IsMIG_Enabled:
     DeviceWatts = DeviceWatts / node.PartitionCount

  # Formula: Base Power / Device Power
  Ratio = MODEL_IDEAL_WATT / DeviceWatts

  Factor = min(1.0, Ratio)

  Score = MAX_SCORE * Factor

  RETURN Score
END FUNCTION
```

Cost, Watt 정보 획득

MIG 전력 반영

전력에 반비례한 점수 산출

③ Workload Affinity

"다중 파드는 어떤 가속기로 배치되어야 하는가?"

➢ 다중 파드 실행 시 job/queue단위로 H/W 가속기 관점에서의 Affinity 점수 반영

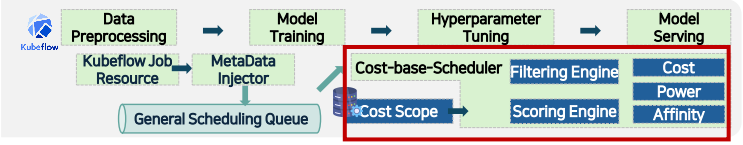
● Pseudocode

```
FUNCTION CalculateAcceleratorAffinity(Node node):
  # Get Strategy
  TopoStrategy = getPodAnnotation(pod, ".topology")

  # Check H/W Peer
  LinkPeers = CountPeersOnSameLink(node,JobName)

  IF TopoStrategy == "NVLINK_PACK":
     Score = LinkPeers * 5.0
  ELSE: # TopoStrategy == "ISOLATE"
     IF LinkPeers == 0:
        Score = 20.0
     ELSE:
        Score = 0.0

  RETURN min(20.0, Score)

END FUNCTION
```

배치 전략 확인

가속기 배치 확인

전략에 따른 점수 산출



성능확인 (1/3) – 실험 정의

사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

실험 내용

- **실험 구성**
  AI Workload 2종에 대한 운영 비용 비교
  >> kube-scheduler vs. cost-base-scheduler
- **클러스터 환경**
  GPU/NPU 혼합형 Kubernetes 클러스터
  >> p10 노드 2대, Kubeflow 기반
- **비교 항목**
  작업 완료까지의 수행 시간, 작업에 사용된 비용
- **H/W 기본 정보**
  - H/W 가속기 시간당 가격 (하드웨어 구매 원가 / 사용 가능 기간 3년 가정))
    - ✓ RTX A6000: 343원/hour
    - ✓ RTX PRO 6000 Blackwell: 685원/hour
    - ✓ NPU Warboy: 305원/hour
  - GPU 전력 소비 (NVIDIA Furiosa)
    - ✓ RTX A6000: 300W
    - ✓ RTX PRO 6000 Blackwell: 600W
    - ✓ NPU Warboy: 50W
  - 전기요금 (한국 산업용 평균가) : 약 172원/kwh

Workload1 – llm-training-pytorch

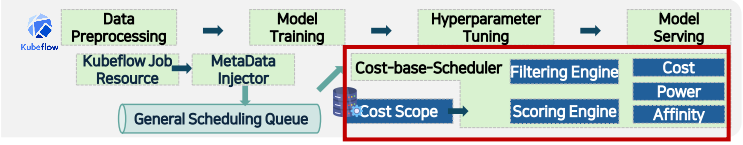
- 이미지 : nvcr.io/nvidia/pytorch:25.04-py3
- 모델명 : gpt2-small
- 수행 파드 수 : 4개
- 작업 종류 : 학습
- 프레임워크 : PyTorch 2.7.0
- 설정값 : Batch Size (16), Dataset Size(6000), Epochs(3) – 학습 시간 5분 이내로 설정)
- Full GPU or MIG Instance 리소스 요청

Workload2 – warboy-vision-npu

- 이미지 : docker.io/library/warboy-vision:latest
- 수행 파드 수 : 1개
- 모델명 : yolov8n
- 작업 종류 : 추론
- 프레임워크 : FuriosaAI (Warboy NPU 전용)
- NPU 전용 워크로드: FuriosaAI Warboy NPU 사용

OPENCLOUD

성능확인 (2/3) – 운영 비용 최적 스케줄링 로그



사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

kubectl logs cost-base-scheduler-79d47c46c4-fkjv8 –n keti

[Manifest]

운영 비용 최적을 위한 알고리즘 3종 적용

[Scoring #1]
gpu-npu-server-02
PRO6000 MIG

[Scoring #2]
gpu-server-01
RTX A6000

[Binding]

```
[Scheduler] Start Scheduling Pod: keti/llm-training-cost-03-worker-0
[Pre-Flight] Detected Annotation: scheduling.gpu.mig-allowed: "true"
# ------------------------------------------------------------------------------------------------------------
[Candidate 1: gpu-npu-server-02] (Spec: Pro6000 x2 / Config: MIG 1/4 Slice Mode)
[Algo: Cost  ] Input: HW($685), Watts(600W), Partitions(4)
[Algo: Cost  ] Calc : Effective_HW(685/4 = 171) + Elec((600/4)/1000 * 172 = 26) = TCO 197 KRW
[Algo: Cost  ] Math : MAX(50.0) - (TCO 197 * Penalty 0.019) ------------------------------------> Score: 46.25
[Algo: Power ] Input: DeviceSpec(600W), Partitions(4) → AllocatedWatts(150W)
[Algo: Power ] Standard: IDEAL_WATT(150W)
[Algo: Power ] Calc : Ratio = Ideal(150) / Allocated(150) = 1.0
[Algo: Power ] Math : MAX(30.0) * Factor(min 1.0, Ratio 1.0) ------------------------------------> Score: 30.00
[Algo: Affin ] Input: Strategy(NVLINK_PACK), LinkPeers(0)
[Algo: Affin ] Logic: Single pod detected (LinkPeers == 0) → Return Max Score for Isolation
[Algo: Affin ] Math : Fixed Score ----------------------------------------------------------------> Score: 20.00
>> Node Total Score with GPU: 96.25
# ------------------------------------------------------------------------------------------------------------
[Candidate 2: gpu-server-01] (Spec: RTX A6000 x2 / Config: Full GPU Mode)
[Algo: Cost  ] Input: HW($343), Watts(300W), Partitions(1)
[Algo: Cost  ] Calc : Effective_HW(343/1 = 343) + Elec((300/1)/1000 * 172 = 52) = TCO 395 KRW
[Algo: Cost  ] Math : MAX(50.0) - (TCO 395 * Penalty 0.025) ------------------------------------> Score: 40.00
[Algo: Power ] Input: DeviceSpec(300W), Partitions(1) → AllocatedWatts(300W)
[Algo: Power ] Standard: IDEAL_WATT(150W)
[Algo: Power ] Calc : Ratio = Ideal(150) / Allocated(300) = 0.5
[Algo: Power ] Math : MAX(30.0) * Factor(min 1.0, Ratio 0.5) ------------------------------------> Score: 15.00
[Algo: Affin ] Input: Strategy(NVLINK_PACK), LinkPeers(0)
[Algo: Affin ] Logic: Single pod detected (LinkPeers == 0) → Return Max Score for Isolation
[Algo: Affin ] Math : Fixed Score ----------------------------------------------------------------> Score: 20.00
>> Node Total Score with GPU: 75.00
# ------------------------------------------------------------------------------------------------------------
[Final Decision]
WINNER: gpu-npu-server-02 (Score 96.25 vs 75.00)
[Target] Physical GPU: NVIDIA RTX 6000 UUID: MIG-5ffbbadf-e256-59fa-a186-40452bbfdfcf (Profile: 1g.12gb)
ACTION: Binding Pod to Node [gpu-npu-server-02]
```

MIG Instance 기반 최종 점수

Full GPU 기반 최종 점수

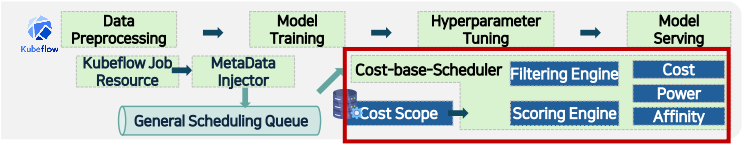
Scheduling 메타데이터를 통한 MIG 배치 가능 확인

시간당 총 소유비용(TCO) 계산

전력에 반비례한 점수 산출

단일 Pod는 Affinity 고려 X

운영 비용 스케줄링 결과 작은 워크로드에 대해 운영 효율이 더 좋은 RTX PRO MIG Instance 선정



사전명세 → 스케줄러 구조 → 스케줄링 알고리즘 → 실험 내용

성능확인 (3/3) - 운영 비용 최적 스케줄링 결과

kube-scheduler GPU 배치 결과

PRO 6000 (MIG)

```
Processes:
  GPU   GI   CI              PID   Type   Process name                  GPU Memory
        ID   ID                                                         Usage
=============================================================================
1   1    3    0          467770      C   python3                         5762MiB
2   1    6    0          467873      C   python3                         5762MiB
```

✓ 2개의 학습 Pod를 RTX Pro6000 MIG Instance2개에 배치

A6000

```
      ID   ID                                                         Usage
=============================================================================
    0   N/A  N/A           2667      G   /usr/lib/xorg/Xorg                 4MiB
3   0   N/A  N/A        2586417      C   python3                         5886MiB
    1   N/A  N/A           2667      G   /usr/lib/xorg/Xorg                 4MiB
4   1   N/A  N/A        2587505      C   python3                         5886MiB
```

✓ 2개의 학습 Pod를 A6000 Full GPU 2개에 배치

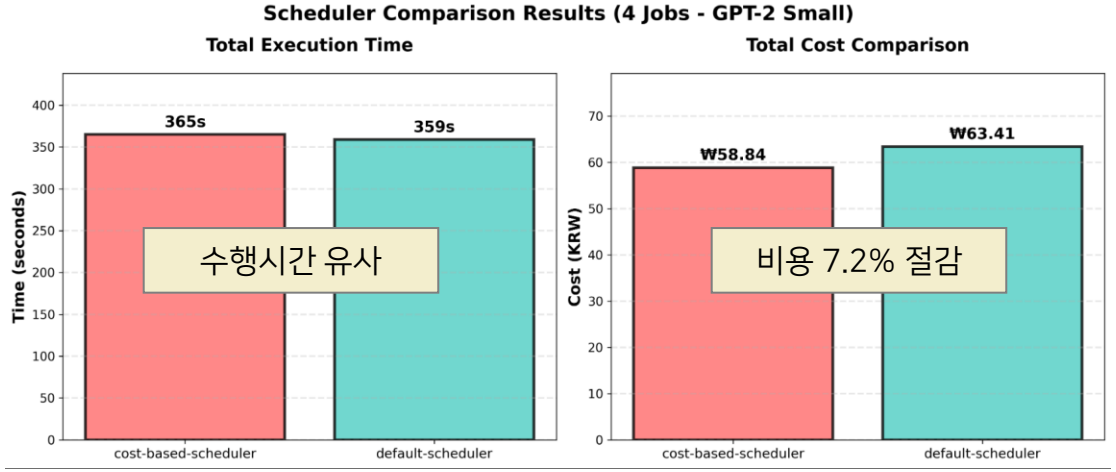
cost-based-scheduler GPU 배치 결과

PRO 6000 (MIG)

```
Processes:
  GPU   GI   CI              PID   Type   Process name                  GPU Memory
        ID   ID                                                         Usage
=============================================================================
1   0    3    0          460883      C   python3                         5762MiB
2   1    3    0          460684      C   python3                         5762MiB
3   1    4    0          460565      C   python3                         5762MiB
4   1    6    0          460836      C   python3                         5762MiB
```

✓ 4개의 학습 Pod에 대해 RTX Pro6000 MIG Instance 4 선정

cost-based-scheduler vs kube-scheduler 결과 비교



✓ kube-scheduler
4개의 학습 Pod에 대해 A6000 Full GPU 2, RTX Pro6000 MIG Instance 2 선정

✓ cost-based-scheduler
4개의 학습 Pod에 대해 RTX Pro6000 MIG Instance 4 선정

➢ 실행시간 : 작업 생성부터 4개 작업의 최종 종료까지의 실행 시간은 거의 동일하게 산출됨

➢ 운영비용 : cost-base-scheduler의 GPU 비용 분석을 통한 배치로 운영 비용 7.2% 절감

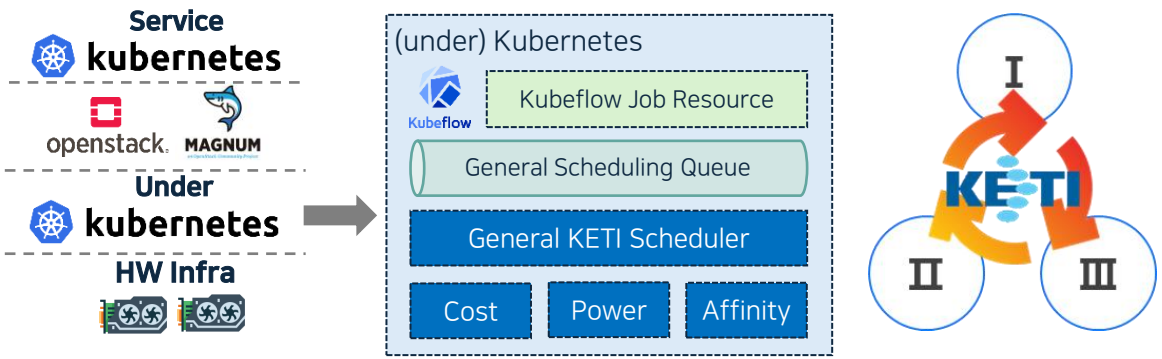
➢ cost-based-scheduler는 GPU 비용 분석을 통해 작업을 저렴한 MIG 인스턴스에 배치함으로써, 실행 시간은 거의 동일하게 유지하면서도 운영 비용을 7.2% 절감하는 효과를 달성

4. 마무리

개발 내용 요약

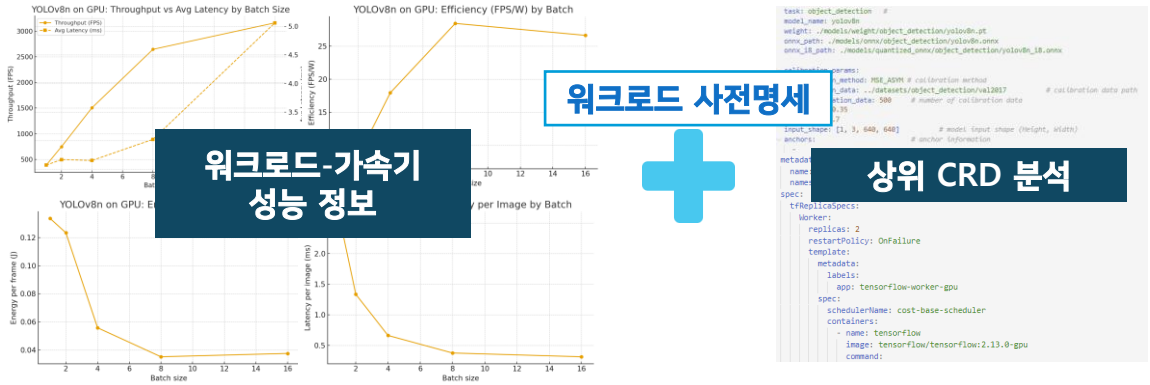
I AI 워크로드 파이프라인 및 검증용 인프라 환경 구축

- Kubeflow 적용 AI 다중 워크로드 검증용 운영 환경 구성
- Kubernetes에서 이기종(GPU·NPU) 반도체 환경 구성



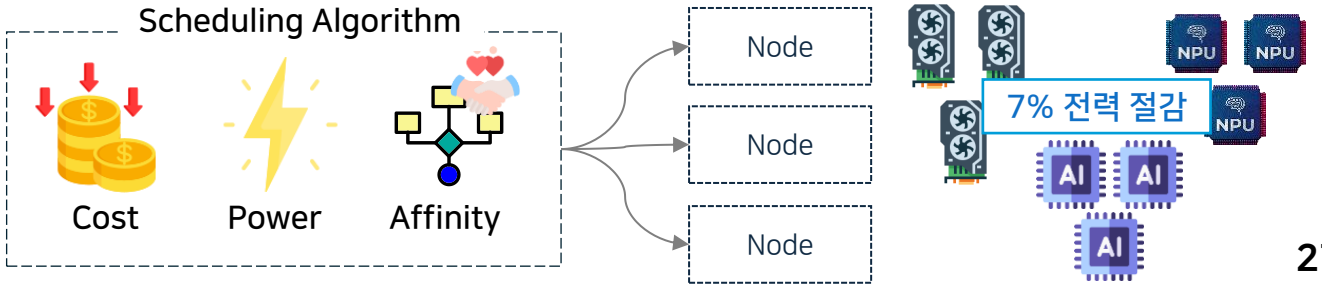
II 워크로드 사전명세 기술

- Kubeflow CRD 분석 기반 워크로드 분석 기술
- 골드 레퍼런스 기반 워크로드-가속기 성능 정보 구축
- 사전명세 기반 스케줄링 메타데이터 자동 삽입



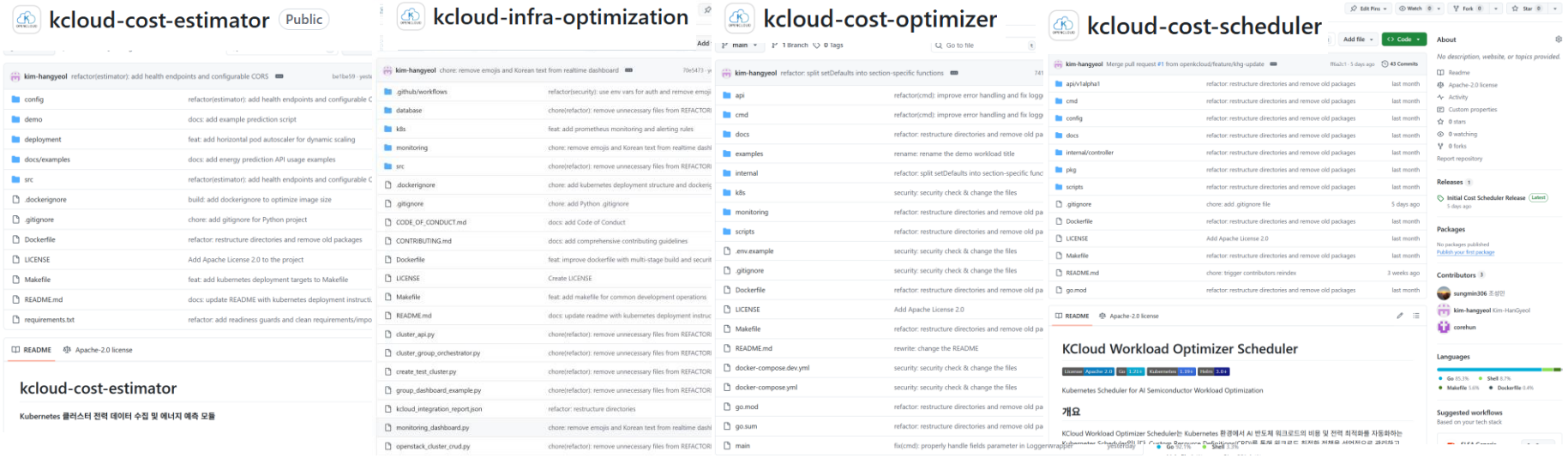
III 운영 비용 최적화 스케줄링 알고리즘

- Cost-Base-Scheduler 구현
- 운용 비용 기반 GPU/NPU 자동 선택 및 배치 최적화
- kube-scheduler 대비 전력 소비 7% 절감 확인



Github 개발 현황 및 계획

- https://github.com/openkcloud

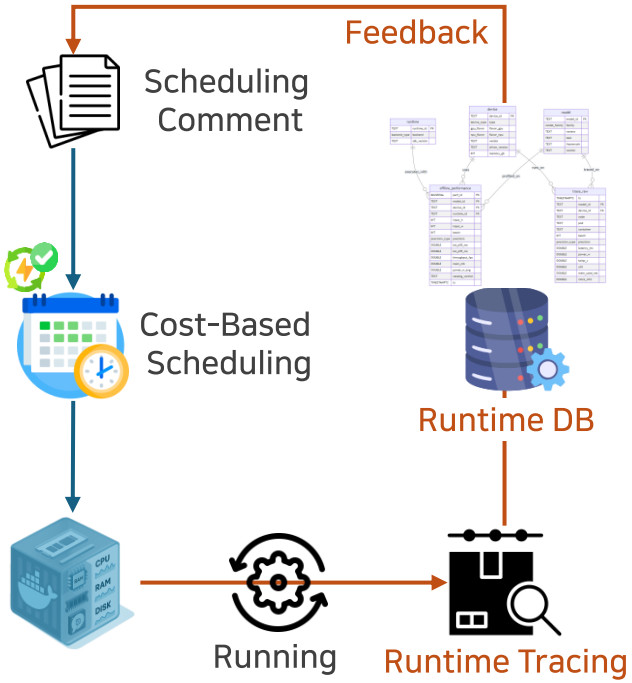


개발 현황 및 개발 계획

향후 확장 계획

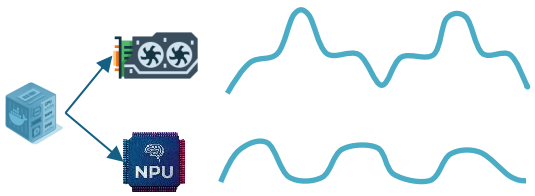
KETI 비용 최적화 스케줄러

워크로드 트레이싱 기반 피드백 사이클

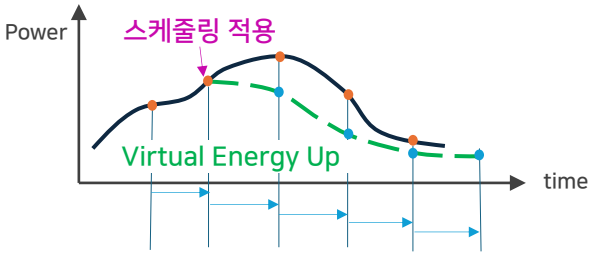


운용 비용 절감 예측 모델

워크로드-하드웨어 비용 패턴 추적



인과적 프로파일링 기반 전력 절감 예측



multi-step forecast

클라우드 서비스 확장

1 STEP — Current Scheduler

Lab Environment	Cost-Power-Affinity
Pod Scheduling	Small Scale GPU/NPU

2 STEP — Large-Scale Service Validation

Real Service Validate	Scheduling Optimize
VM+Pod Unified	Enterprise Pilot

3 STEP — Production Scale K-Cloud Platform

K-Cloud Validation	Multi-Tenant
Public Cloud	Private Cloud

차세대 운영 비용 최적화 스케줄러

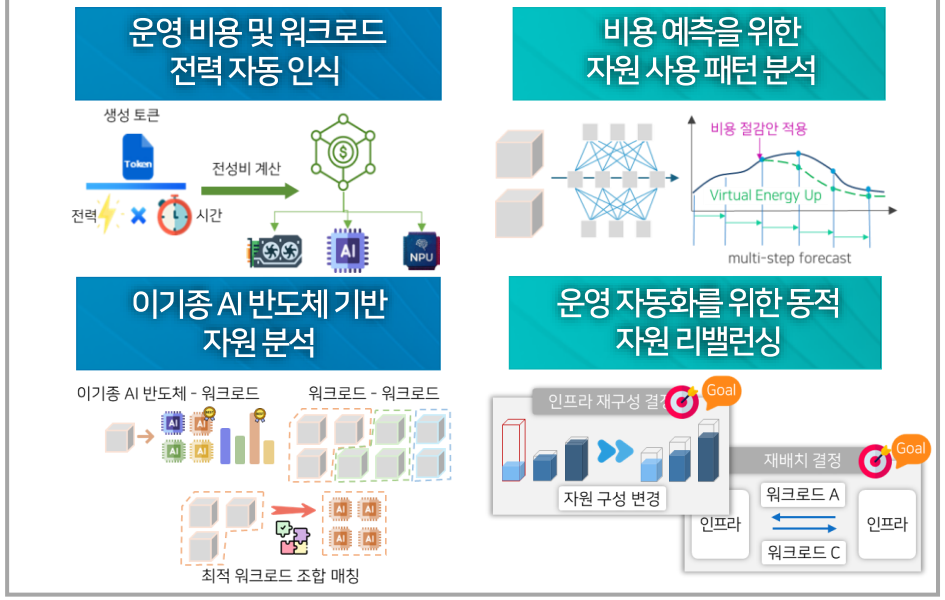
	자원 기반 스케줄러	정책 기반 스케줄러	운용 효율 기반 스케줄러
			자동화 기반 비용 절감
		배치 제약 친화성 규칙	운영 비용 및 전성비 인식
입력 신호	requests/limits	CRD 제약/큐 상태	사전명세 + 트레이닝 데이터
자원 결정 기준	자원 충족	정책 만족	효율 지표
자동화 수준	낮음	중간	높음

운영 비용 최적화 스케줄링

- ✓ 운영 비용 및 워크로드 전력 자동 인식 + 이기종 AI 반도체 기반 자원 조합 분석
- ✓ 비용 예측을 위한 자원 사용 패턴 분석 + 운영 자동화를 위한 동적 자원 리밸런싱

운영 비용 최적화 스케줄링

- AI 워크로드 기반 비용 최적 운영 정책 제공
- 비용 예측 및 전력 최적화 파이프라인



감사합니다.